

1 Preliminary Questions

1.1 Geometric interpretation of the similarity

The loss function is:

$$\mathcal{L} = -\mathbf{1}_{c \in C^+} \log \sigma(\mathbf{c}^\top \mathbf{w}) - \mathbf{1}_{c \in C^-} \log(1 - \sigma(\mathbf{c}^\top \mathbf{w}))$$

where $\mathbf{w} = E_w(w)$ and $\mathbf{c} = E_C(c)$.

- For $c \in C^+$, minimizing $-\log \sigma(\mathbf{c}^\top \mathbf{w})$ amounts to maximizing $\sigma(\mathbf{c}^\top \mathbf{w})$, and therefore maximizing the dot product $\mathbf{c}^\top \mathbf{w}$.
- For $c \in C^-$, minimizing $-\log(1 - \sigma(\mathbf{c}^\top \mathbf{w}))$ amounts to minimizing $\sigma(\mathbf{c}^\top \mathbf{w})$, and therefore minimizing the dot product $\mathbf{c}^\top \mathbf{w}$.

Geometric interpretation. Since the sigmoid function is monotonic, maximizing $\sigma(\mathbf{c}^\top \mathbf{w})$ corresponds to increasing the alignment between vectors. As a result, positive pairs tend to have vectors pointing in similar directions (small angle), while negative pairs are pushed apart, leading to orthogonal or even opposite directions. Consequently, words that frequently co-occur are embedded close to each other, whereas unrelated words are separated in the vector space.

1.2 Relation to *Chopra et al.*

Contrastive learning, as introduced by *Chopra et al.* [1], aims to learn a similarity function from pairs of inputs. The objective is to map similar inputs to nearby representations, while dissimilar inputs are pushed further apart in the embedding space.

This principle is directly reflected in Word2Vec with negative sampling, where positive word-context pairs correspond to similar pairs, and randomly sampled words serve as negative pairs.

The correspondence with the formulation in *Chopra et al.* is as follows:

- The label Y indicates whether a pair is dissimilar; in Word2Vec, this corresponds to whether $c \in C^-$.
- The embedding function E_W maps inputs to vector representations. In Word2Vec, this role is played by the embedding tables E_w and E_C , which map center words and context words to vectors.
- However, unlike the original formulation, E_W is not a shared neural network applied to both inputs. Instead, Word2Vec uses two separate embedding matrices and computes similarity using a dot product rather than a learned distance such as the Euclidean distance.
- The losses L_G and L_I correspond respectively to $-\log \sigma(\mathbf{c}^\top \mathbf{w})$ for positive pairs and $-\log(1 - \sigma(\mathbf{c}^\top \mathbf{w}))$ for negative pairs.

2 Implementation Summary

2.1 Data Preprocessing

A subset of 10,000 IMDB reviews was shuffled and tokenized using `bert-base-uncased` (vocabulary size $|\mathcal{V}| = 30,522$), with sequences truncated to a maximum length of 256 tokens. The dataset was then split into 8,000 training and 2,000 validation documents. Since Word2Vec is an unsupervised method, labels are not used. The function `extract_words_contexts` constructs training pairs (w_i, C_i^+) , where C_i^+ consists of the $R = 5$ words surrounding position i . To ensure consistent context size and avoid padding, positions near document boundaries ($i < R$ or $i \geq \text{len}(d) - R$) are skipped. This removes only a small fraction of tokens (approximately 3.9%). Negative samples C^- are drawn uniformly at batch time, with $2KR$ negatives per center word. This on-the-fly sampling ensures that each epoch sees different negative examples, effectively acting as a form of data augmentation. After preprocessing, the dataset contains **1,573,630** training pairs and **390,063** validation pairs.

2.2 Model and Training

The Word2Vec model uses two embedding tables: one for center words (E_w) and one for context words (E_C), both in $\mathbb{R}^{|\mathcal{V}| \times d}$ and initialized uniformly in $\mathcal{U}[-0.1, 0.1]$.

During the forward pass, similarities between words and their contexts are computed using batched matrix multiplication (bmm), followed by a sigmoid function. The model is trained using a binary cross-entropy loss:

$$\mathcal{L} = -\frac{1}{B \cdot 2R} \sum \log \text{pos_scores} - \frac{1}{B \cdot 2KR} \sum \log(1 - \text{neg_scores}).$$

Training is performed using the Adam optimizer with a learning rate of 10^{-3} . Two configurations were evaluated:

Config	d	R	K	B	E	Train loss	Val. acc.
Small	64	5	5	512	5	0.4342	91.19%
Large	128	5	5	512	10	0.3740	91.11%

Table 1: Final metrics for both configurations.

Both models achieve over 91% validation accuracy, indicating that the embeddings effectively capture word co-occurrence patterns. However, their generalisation behaviour differs. The small model shows stable validation performance after epoch 3, with only mild overfitting. In contrast, the large model exhibits a clear divergence between training and validation loss: while training loss continues to decrease, validation loss increases after epoch 3, reaching 0.525 at epoch 10. This suggests overfitting beyond epoch 4. Overall, a moderate embedding dimension ($d = 64$) with a limited number of epochs ($E \approx 3-5$) provides the best trade-off between performance and generalisation on this dataset. Model checkpoints are saved using the format: `model_dim-{d}_radius-{R}_ratio-{K}_batch-{B}_epoch-{E}.ckpt`.

3 Few-Shot Sentiment Classification

3.1 Objective

The objective of this experiment is to evaluate whether pretrained Word2Vec embeddings can improve the performance of a sentiment classifier in a low-resource setting. In order to simulate a realistic few-shot scenario, the model is trained on only 64 labeled examples. This setting is particularly challenging, as the model must generalize from very limited supervision.

3.2 Method

A simple feed-forward neural network is used as a classifier on top of word embeddings. Three different setups are considered in order to assess the impact of pretraining and model capacity. In the first case, embeddings are randomly initialized and learned only from the few-shot dataset, providing a baseline. In the second case, embeddings are initialized using a pretrained Word2Vec model with dimension $d = 64$. In the third case, a larger pretrained model with $d = 128$ is used.

In all configurations, the model is trained for 10 epochs using the Adam optimizer with a learning rate of 10^{-2} . During training, standard metrics are monitored, including training loss, training accuracy, validation loss, and validation accuracy. Detailed training curves (loss and accuracy) are provided in the notebook `imdb_few_shots`.

3.3 Training Results

Across all configurations, the training dynamics are very similar. The training loss decreases rapidly and becomes close to zero after only a few epochs, while the training accuracy reaches 100% extremely quickly (typically before epoch 5). This behavior indicates that the model is able to perfectly fit the small training dataset.

On the validation set, performance improves more gradually. Depending on the configuration, the validation accuracy increases from approximately 51–54% at the first epoch to values between 66% and 70% at convergence. However, validation loss stabilizes early and may slightly increase after a few epochs, suggesting the onset of overfitting.

3.4 Analysis

The results clearly highlight the difficulty of the few-shot setting. Due to the extremely small number of training examples (64), the model quickly memorizes the dataset, as evidenced by the rapid convergence to 100% training accuracy.

In contrast, generalization remains limited. Although validation accuracy improves significantly during the first epochs, it quickly saturates, and validation loss stops decreasing. This gap between training and validation performance is a clear indicator of overfitting.

A more detailed comparison between configurations shows that pretrained embeddings can provide a measurable benefit, but this effect depends on the embedding dimension. The best validation accuracies obtained are:

- Random initialization (Figure 1): 67.4%
- Word2Vec ($d = 64$) (Figure 2): 66.8%
- Word2Vec ($d = 128$) (Figure 3): 69.6%

The larger pretrained model ($d = 128$) achieves the best performance, outperforming the random baseline by more than 2 points. This indicates that semantic information learned during Word2Vec training can effectively transfer to the downstream task when the embedding space is sufficiently expressive.

However, the smaller pretrained model ($d = 64$) does not outperform the baseline. This suggests that pretraining alone is not sufficient: the representation capacity must also be large enough to capture useful semantic features. If the embedding dimension is too small, the transferred information may be limited.

Overall, these results highlight a trade-off between pretraining and model capacity. While pretrained embeddings can improve performance, their effectiveness depends on the size of the representation and remains constrained by the severe lack of labeled data.

3.5 Ablation Study

We analyze the impact of pretrained embeddings and embedding dimension on the few-shot classification task.

Model	Val. acc. (best)
Random initialization ($d = 128$)	67.4%
Word2Vec ($d = 64$)	66.8%
Word2Vec ($d = 128$)	69.6%

Table 2: Few-shot performance depending on initialization and embedding dimension.

The ablation study confirms that pretrained embeddings improve performance when combined with a sufficiently large embedding dimension. In particular, the $d = 128$ pretrained model achieves the highest accuracy (69.6%), demonstrating effective knowledge transfer from the unsupervised Word2Vec training.

However, the results also show that pretraining is not always beneficial. The $d = 64$ pretrained model performs slightly worse than the random baseline, indicating that limited representation capacity can restrict the usefulness of pretrained features.

In all configurations, training accuracy quickly reaches 100%, while validation accuracy saturates. This confirms that the main limitation is not optimization, but generalization due to the extremely small training set. Increasing model capacity helps, but does not fully overcome this limitation.

These observations emphasize that both pretraining and model size are important factors, but that additional data or regularization techniques would likely be required to achieve further improvements in this few-shot setting.

4 Conclusion

This homework presented a complete from-scratch implementation of Word2Vec with negative sampling in PyTorch, including careful design choices such as a truncated-window strategy, on-the-fly negative sampling, and the use of separate embedding tables. The resulting embeddings achieve over 91% accuracy on the binary co-occurrence task, showing that the model successfully captures word relationships.

An important observation is that increasing model capacity ($d = 128$, more epochs) does not always improve generalisation on a small corpus, as the larger model quickly overfits. This highlights the need for careful training strategies such as early stopping when working with limited data.

In the few-shot setting (64 training examples), the best model reaches 69.6% validation accuracy. Pretrained embeddings provide a benefit only when the embedding dimension is sufficiently large ($d = 128$), while smaller embeddings ($d = 64$) do not outperform random initialization. This demonstrates that both pretraining and representation capacity are crucial, but remain limited by the scarcity of labeled data.

References

- [1] Sumit Chopra, Raia Hadsell, and Yann LeCun. “Learning a Similarity Metric Discriminatively, with Application to Face Verification”. In: *Proceedings of the 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR’05) - Volume 1 - Volume 01*. CVPR ’05. USA: IEEE Computer Society, 2005, pp. 539–546. ISBN: 0769523722. DOI: [10.1109/CVPR.2005.202](https://doi.org/10.1109/CVPR.2005.202).

Appendix

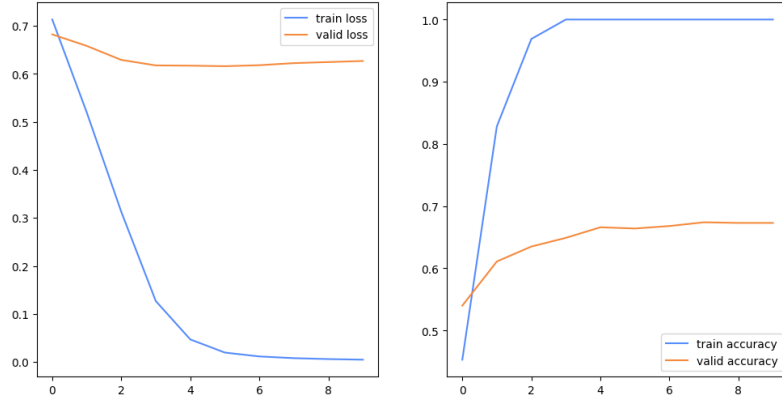


Figure 1: Few-shot training curves with random initialization (best validation accuracy: 67.4%). Left: training and validation loss. Right: training and validation accuracy.

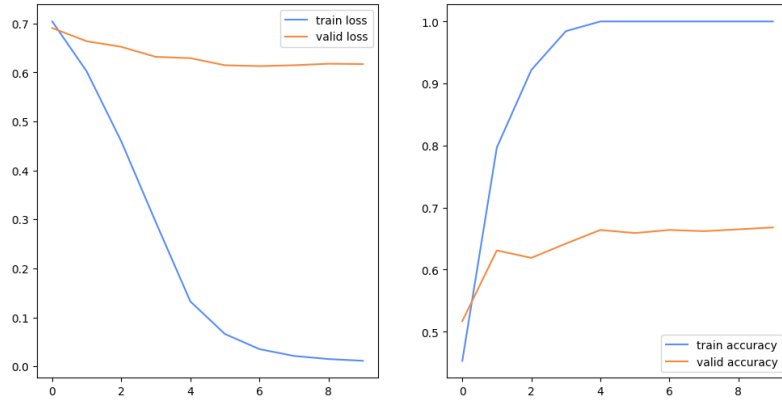


Figure 2: Few-shot training curves with pretrained Word2Vec embeddings ($d = 64$) (best validation accuracy: 66.8%). Left: training and validation loss. Right: training and validation accuracy.

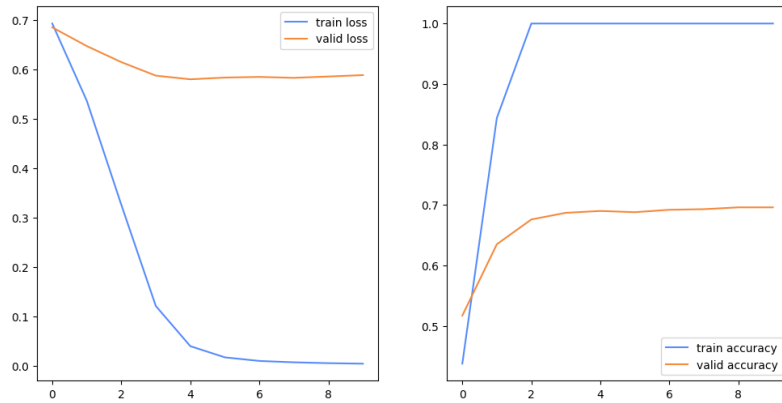


Figure 3: Few-shot training curves with pretrained Word2Vec embeddings ($d = 128$) (best validation accuracy: 69.6%). Left: training and validation loss. Right: training and validation accuracy.